

PyNumDiff 2.0: Practical Numerical Differentiation for Noisy Data

Pavel Komarov^{1¶}, Floris van Breugel², Maria Protogerou⁴, and J. Nathan Kutz³

¹ Department of Electrical and Computer Engineering, University of Washington, USA ² Department of Mechanical Engineering, University of Nevada, Reno, USA ³ Autodesk Research, London, UK ⁴ Department of Applied Mathematics, University of Washington, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Prashant Jha](#) 

Submitted: 02 April 2026

Published: unpublished

License

Authors of papers retain copyright, and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Derivatives of measured data are a prerequisite across science and engineering: identifying governing equations, designing controllers, and processing sensor streams alike. The textbook remedy, finite differencing, amplifies noise as $1/\Delta t$ and deteriorates rapidly as data grows noisier or more finely sampled. Smoothing before differencing helps, but algorithm choice and tuning substantially affect the result, and no single approach wins universally.

PyNumDiff is an open-source Python package consolidating a broad suite of numerical differentiation methods under a unified API. Seven algorithm families are implemented: (1) prefiltering followed by finite difference calculation; (2) iterated finite differencing; (3) polynomial fitting ([Savitzky & Golay, 1964](#)); (4) spectral, radial basis function, and wavelet fitting; (5) total variation regularization ([Chartrand, 2011](#)); (6) Kalman smoothing ([Kalman, 1960](#); [Rauch et al., 1965](#)); and (7) local approximation with linear models. Ahnert & Abel (2007) provides a useful taxonomy of these families, distinguishing local methods (which estimate derivatives from a surrounding window) from global methods that fit the entire signal at once. All PyNumDiff methods return a matched pair ($\hat{x}$, $\hat{dx/dt}$). A companion paper ([Komarov et al., 2025](#)) benchmarks all methods across test signals and guides selection for different application scenarios.

Statement of Need

Derivative estimation from noisy measurements arises throughout experimental science, data-driven modeling, system identification, and control. The field offers a diverse ecosystem of specialized algorithms, each with different strengths in outlier robustness, computational cost, irregular sampling, or missing observations, but no consolidated home. Without one, practitioners are left assembling solutions piecemeal from disparate packages ([Floris van Breugel et al., 2022](#)).

PyNumDiff addresses this gap. Its unified interface lets users compare methods on the same data, exploit specialized capabilities, and select hyperparameters without ground-truth derivatives. Derivative estimation trades data fidelity against smoothness; PyNumDiff frames this as a multi-objective optimization, finding hyperparameter settings that minimize a weighted combination of the two. A single scalar weight tvgamma steers between objectives in the absence of ground-truth, or, when a ground-truth derivative is available, it can serve it as the optimization target. This framework spans all methods, enabling principled comparison and selection ([Floris van Breugel et al., 2020](#)). A natural and growing application is SINDy ([Brunton et al., 2016](#)), which discovers governing equations by regressing measured derivatives, making the quality of those estimates a direct determinant of model accuracy.

State of the Field

Relevant Python tools exist, but none covers PyNumDiff's breadth. `numpy.gradient` and `scipy.signal.savgol_filter` (Virtanen et al., 2020) handle only a sliver of the method space; `findiff` offers high-order finite difference stencils suited to clean simulation data, not noisy measurements. Historically, practitioners have had to stitch together PyKalman, PyDMD, and standalone TVR scripts (Chartrand, 2011) with no shared API or principled way to compare results. The derivative package (Goldschmidt, 2021) overlaps substantially but lacks multidimensional support, NaN handling, and hyperparameter optimization. No existing package spans PyNumDiff's seven method families with a consistent interface.

The original PyNumDiff publication (Florin van Breugel et al., 2022) established the core method set and optimization framework. This version substantially revises it: the taxonomy was reorganized and corrected; the interface unified around explicit keyword arguments; the hyperparameter optimizer was substantially improved; test coverage expanded; and four capabilities were added throughout: multidimensional data, irregular sample spacing, missing observations, and circular domains.

Software Design

Package design. All differentiation methods share the call signature

```
x_hat, dxdt_hat = method(x, dt_or_t, **params)
```

where `x` is a NumPy array (Harris et al., 2020) of measurements; `dt_or_t` is either a scalar step size or an array of sample locations; and keyword arguments configure the method, making calls self-documenting. Prior positional signatures are preserved with deprecation warnings.

Software architecture. PyNumDiff is organized into seven method modules plus shared `utils` and `optimize` modules in a flat structure. Where strong alternatives exist, PyNumDiff delegates rather than reimplements: SciPy (Virtanen et al., 2020) provides spline fitting, Savitzky-Golay filtering, and signal processing routines; NumPy (Harris et al., 2020) provides the FFT; PyWavelets (Lee et al., 2019) provides the discrete wavelet transform for `waveletdiff`; CVXPY (Diamond & Boyd, 2016) handles convex optimization for `robustdiff` and `tvrdiff` as an optional dependency. The public `kalman_filter` and `rts_smooth` primitives let users with known dynamics bypass `rtstdiff`'s constant-derivative model; an `innovation_fn` hook extends the filter to non-Euclidean spaces.

Method capabilities. All non-deprecated methods support multidimensional data via `axis`; Table 1 lists additional specialized capabilities.

Table 1: Specialized capabilities by method.

Method	Variable step	Missing Data	Outlier Robust	Circular Domain
<code>kerneldiff</code>				
<code>finitediff</code>				
<code>polydiff</code>	✓	✓		
<code>savgoldiff</code>				
<code>splinediff</code>	✓	✓		
<code>spectraldiff</code>				
<code>rbfdiff</code>	✓			
<code>waveletdiff</code>				
<code>tvrdiff</code>			✓	
<code>rtstdiff</code>	✓	✓		✓
<code>robustdiff</code>	✓	✓	✓	
<code>lineardiff</code>				

73 **Variable sample spacing.** Methods that support variable step size accept an array of sample
74 locations in place of a scalar step size. For Kalman-based methods, this means computing the
75 discrete-time transition matrix via matrix exponential at each actual sample interval rather
76 than extracting a fixed Δt from the first two samples.

77 **Missing data.** NaN-valued entries are treated as missing observations: excluded from fitting,
78 imputed from the model. This supports sensors that occasionally drop samples without any
79 user preprocessing.

80 **Outlier robustness.** `robustdiff` replaces the quadratic Kalman cost with Huber loss terms
81 on both measurement and process residuals, following Aravkin et al. (2013), with CVXPY
82 (Diamond & Boyd, 2016) as the optimization backend; the sparse problem formulation scales
83 linearly with signal length. `tvrdiff` similarly applies Huber loss on data fidelity; its total
84 variation penalty on the derivative additionally promotes piecewise-smooth solutions for signals
85 with abrupt transitions.

86 **Circular and wrapped domains.** `rtstdiff` accepts `circular=True` for quantities like angles on a
87 periodic domain. Innovation residuals are wrapped to $[-\pi, \pi]$ before each Kalman update via
88 an `innovation_fn` hook, and `x_hat` is returned in the same range, avoiding the large spurious
89 spikes naive smoothers produce when a signal crosses the $\pm\pi$ boundary.

90 **Hyperparameter optimization.** `pynumdiff.optimize` minimizes the weighted combination
91 described above (Floris van Breugel et al., 2020). The smoothness weight `tvgamma` can be
92 initialized from the signal's estimated cutoff frequency f_c via

$$\text{tvgamma} = \exp(-1.6 \ln f_c - 0.71 \ln \Delta t - 5.1).$$

93 Four improvements are made in this version: evaluations are cached to avoid redundant
94 work; the loss is robustified via Huber penalty so outliers do not bias parameter selection; the
95 Kalman parameter space is reduced from two independent noise variances to their log-ratio,
96 the only salient factor (Komarov et al., 2025); and categorical and boolean hyperparameters
97 are supported natively, enabling joint optimization over discrete choices such as derivative
98 order alongside continuous parameters.

99 **Testing and continuous integration.** The test suite validates all methods against analytic
100 functions with known derivatives, covering noiseless and noisy cases across the full expected
101 accuracy range. Care was taken to avoid tautological tests where the implementation directly
102 determines the expected result. Tests run automatically on every push and pull request via
103 GitHub Actions, with line coverage tracked via Coveralls.

104 Research Impact

105 The original PyNumDiff paper (Floris van Breugel et al., 2022) has been applied in experimental
106 biology (flight kinematics from motion capture), control engineering (observer design), and
107 data-driven dynamics identification via SINDy (Brunton et al., 2016). The present version ships
108 under the MIT License, is available on PyPI (`pip install pynumdiff`), and is accompanied by
109 Jupyter notebook tutorials and full Sphinx API documentation at pynumdiff.readthedocs.io. The
110 companion Taxonomy paper (Komarov et al., 2025), submitted to the Journal of Computational
111 Physics, provides the theoretical underpinning and benchmarks all included methods. The
112 PySINDy project (Silva et al., 2020) maintains its own differentiation submodule substantially
113 overlapping with PyNumDiff's capabilities; integration discussions are ongoing.

114 AI Usage Disclosure

115 This paper was drafted with assistance from Claude Sonnet 4.6 and Claude Opus 4.8 (Anthropic),
116 which also implemented successive code revisions to address recent issues and author feedback.

117 All outputs were reviewed and further edited by hand, and the authors take full responsibility
118 for accuracy.

119 Acknowledgements

120 The authors thank Yuying Liu and Bingni W. Brunton for their contributions to the original
121 PyNumDiff package (Floris van Breugel et al., 2022), and Sasha Aravkin for discussions on
122 convex optimization techniques that informed the robust differentiation methods. This work
123 was supported by the NSF AI Institute in Dynamic Systems (grant number 2112085).

124 References

- 125 Ahnert, K., & Abel, M. (2007). Numerical differentiation of experimental data: Local
126 versus global methods. *Computer Physics Communications*, 177(10), 764–774. <https://doi.org/10.1016/j.cpc.2007.03.009>
127
- 128 Aravkin, A. Y., Burke, J. V., & Pillonetto, G. (2013). Optimization viewpoint on Kalman
129 smoothing with applications to robust and sparse estimation. *Journal of Machine Learning*
130 *Research*, 14, 2513–2558. https://doi.org/10.1007/978-3-642-38398-4_8
- 131 Breugel, Floris van, Liu, Y., Brunton, B. W., & Kutz, J. N. (2022). PyNumDiff: A Python
132 package for numerical differentiation of noisy time-series data. *Journal of Open Source*
133 *Software*, 7(71), 4078. <https://doi.org/10.21105/joss.04078>
- 134 Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from
135 data by sparse identification of nonlinear dynamical systems. *Proceedings of the National*
136 *Academy of Sciences*, 113(15), 3932–3937. <https://doi.org/10.1073/pnas.1517384113>
- 137 Chartrand, R. (2011). Numerical differentiation of noisy, nonsmooth data. *ISRN Applied*
138 *Mathematics*, 2011, 164564. <https://doi.org/10.5402/2011/164564>
- 139 Diamond, S., & Boyd, S. (2016). CVXPY: A Python-embedded modeling language for convex
140 optimization. *Journal of Machine Learning Research*, 17(83), 1–5.
- 141 Goldschmidt, A. (2021). *Derivative: Numerical differentiation in Python*. <https://github.com/andgoldschmidt/derivative>
142
- 143 Harris, C. R., Millman, K. J., van der Walt, S. J., & others. (2020). Array programming with
144 NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 145 Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of*
146 *Basic Engineering*, 82(1), 35–45. <https://doi.org/10.1115/1.3662552>
- 147 Komarov, P., Breugel, F. van, & Kutz, J. N. (2025). *A taxonomy of numerical differentiation*
148 *methods*. <https://arxiv.org/abs/2512.09090>
- 149 Lee, G. R., Gommers, R., Wasilewski, F., Wohlfahrt, K., & O’Leary, A. (2019). PyWavelets:
150 A Python package for wavelet analysis. *Journal of Open Source Software*, 4(36), 1237.
151 <https://doi.org/10.21105/joss.01237>
- 152 Rauch, H. E., Tung, F., & Striebel, C. T. (1965). Maximum likelihood estimates of linear
153 dynamic systems. *AIAA Journal*, 3(8), 1445–1450. <https://doi.org/10.2514/3.3166>
- 154 Savitzky, A., & Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified
155 least squares procedures. *Analytical Chemistry*, 36(8), 1627–1639. <https://doi.org/10.1021/ac60214a047>
156
- 157 Silva, B. M. de, Champion, K., Quade, M., Loiseau, J.-C., Kutz, J. N., & Brunton, S. L. (2020).
158 PySINDy: A Python package for the sparse identification of nonlinear dynamics from data.
159 *Journal of Open Source Software*, 5(49), 2104. <https://doi.org/10.21105/joss.02104>

- 160 van Breugel, Floris, Kutz, J. N., & Brunton, B. W. (2020). Numerical differentiation of noisy
161 data: A unifying multi-objective optimization framework. *IEEE Access*, 8, 196865–196877.
162 <https://doi.org/10.1109/ACCESS.2020.3034077>
- 163 Virtanen, P., Gommers, R., Oliphant, T. E., & others. (2020). SciPy 1.0: Fundamental
164 algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
165

DRAFT